# **Lecture 23:** Regularization; Unsupervised Learning: Clustering

**CMSC/DATA 320: Introduction to Data Science**

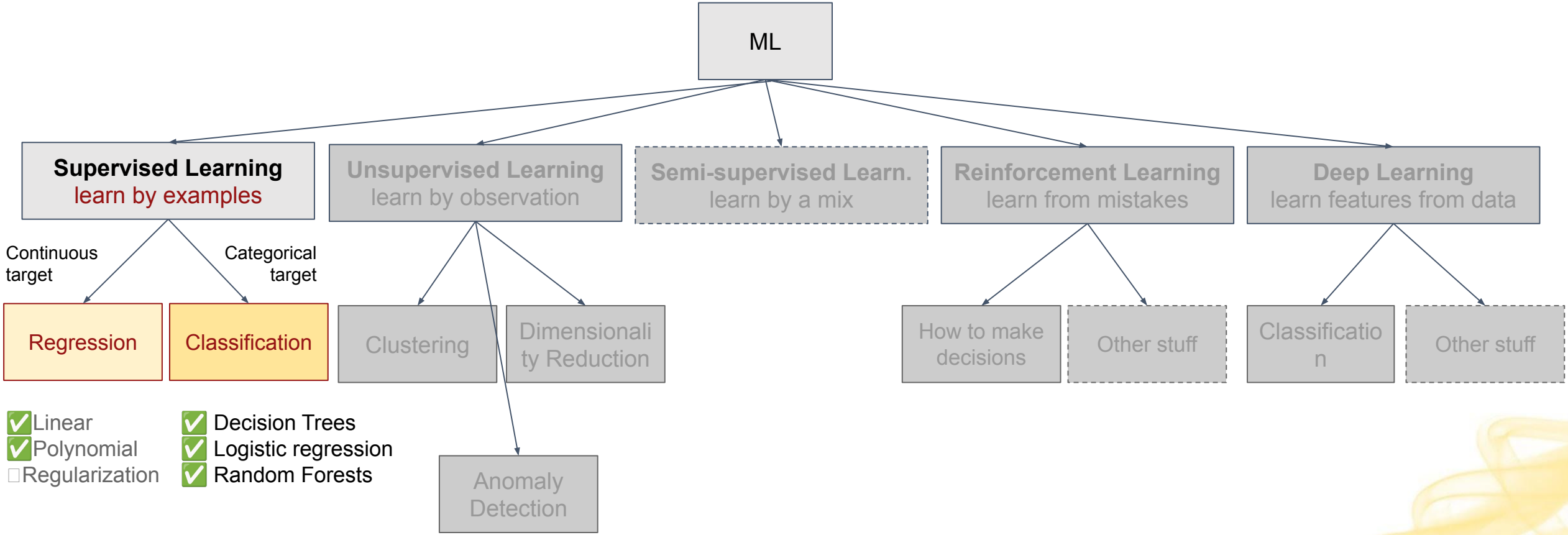
**Lecturer: Anna Evtushenko (annae@umd.edu)**

# Announcements

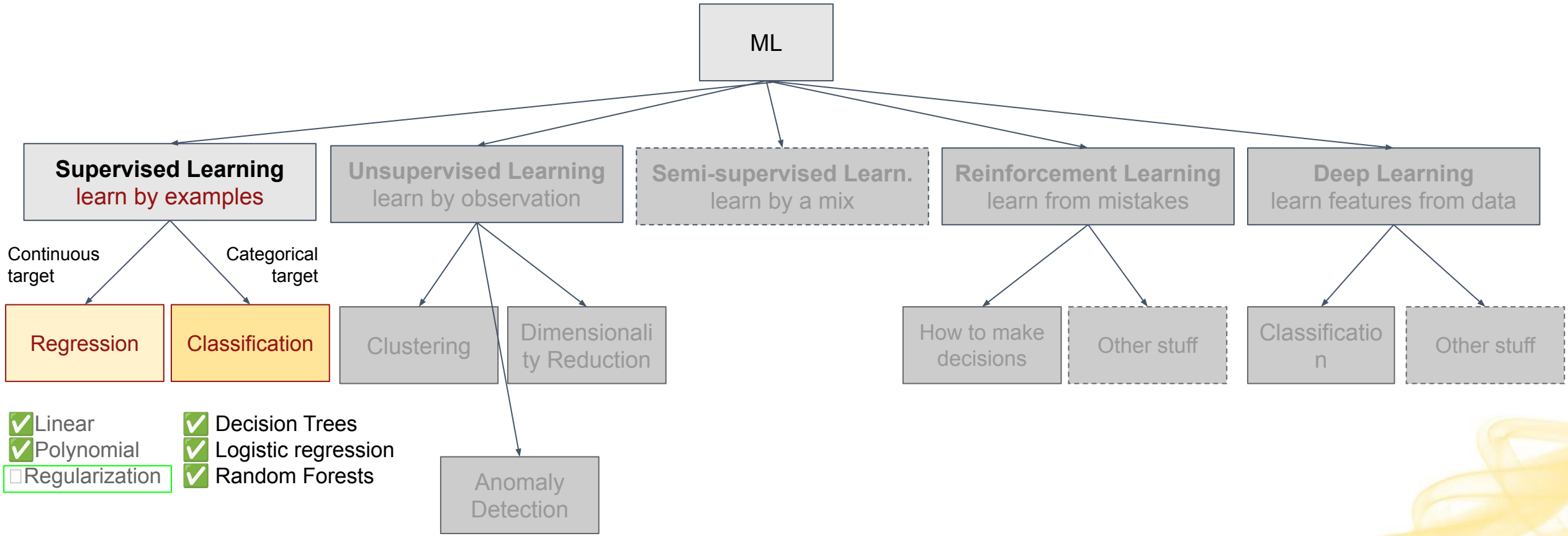
- PA6, WA6 due Sun April 26
  - Best to know this week's material (Mon and perhaps Wed)
- Q12 grades out
- WA5 grades out
- PA5 grades out

# Regularization

# Main categories of Machine Learning



# Main categories of Machine Learning



# Regularization

**Definition**: Regularization is defined as **minimizing or eliminating parameters** from a model in order to obtain a **simpler prediction problem**.

- Helps with **interpretability** and reduces **overfitting**
- We'll discuss regularization in the context of **regression**, but it is a **broad concept** that applies across many areas of machine learning

# Regularization in Regression

The most common form of regularization is one in which we modify the linear regression cost function to add a complexity term that **penalizes certain values of the coefficients** (weights) we are learning.

- Idea: Penalize coefficients whose values are “large”

# Motivation for Regularization

If we have many features, the model might assign weights to all of them, leading potentially to very small weights for many features, and in general leading to overfitting.

We want to include a weight at all if the feature is important (i.e. if the weight would be high).

# Coefficient Penalties

**Three flavors**:

1. Penalize using the **absolute value** (magnitude) of the coefficients
   - Causes some coefficients to be **0**
   - Called the **L1 Norm**
   - Algorithm is called LASSO (Least Absolute Shrinkage and Selection Operator)
2. Penalize using the **square** of the coefficients (most common)
   - Causes all coefficients to be **small**
   - Called the **L2 norm**
   - Algorithm is called Ridge
3. A **hybrid** of the above two
   - Algorithm is called Elastic-Net

# Regression Cost Functions with Regularization

Before, with multiple regression: $J(\beta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{k}\beta_j x_{ij}\right)^2$

1. **LASSO** (L1 norm)

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{k}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=i}^{k}|\beta_j|$$

2. **Ridge** (L2 norm)

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{k}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=i}^{k}\beta_j{}^2$$

3. **Elastic-Net** (hybrid)

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{k}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=i}^{k}\beta_j{}^2 + |\beta_j|$$

# Ensuring a feature is useful

Regardless of which regularization technique we use, the core idea is that a coefficient doesn't merely need to result in a better solution, **it needs to be sufficiently beneficial to justify its inclusion**.

- What "sufficiently beneficial" means is determined by the regularization constant $\boldsymbol{\lambda}$ (sometimes called α (sklearn) or the regularization strength)
  - Lambda ($\lambda$) is a **hyperparameter** we tune to control the bias-variance tradeoff
  - Determines how much we penalize for large coefficients
  - Setting $\lambda$ to zero removes the regularization

# Use of LASSO (L1) for feature selection

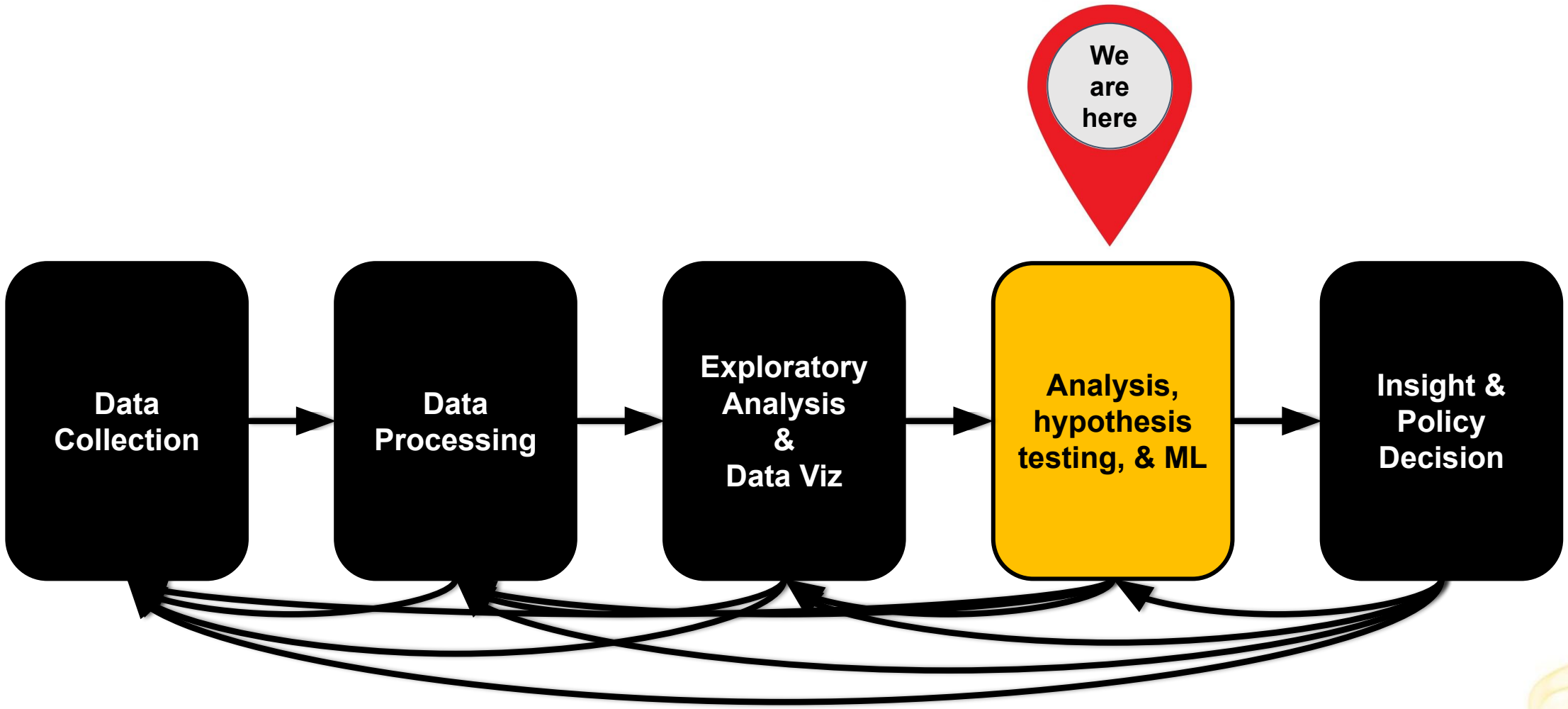
- Because it attempts to eliminate features, LASSO can be a good method of selecting the most significant features
  - Particularly important for prescriptive data science
  - Generally performs better than Forward Selection and Backwards elimination, but less computationally efficient
- However, this approach should be taken with care
  - If multiple features are highly correlated, LASSO may essentially pick one to keep at random
  - Sometimes, features simply have a small but very real contribution
  - Need to use cross-validation to choose a good lambda

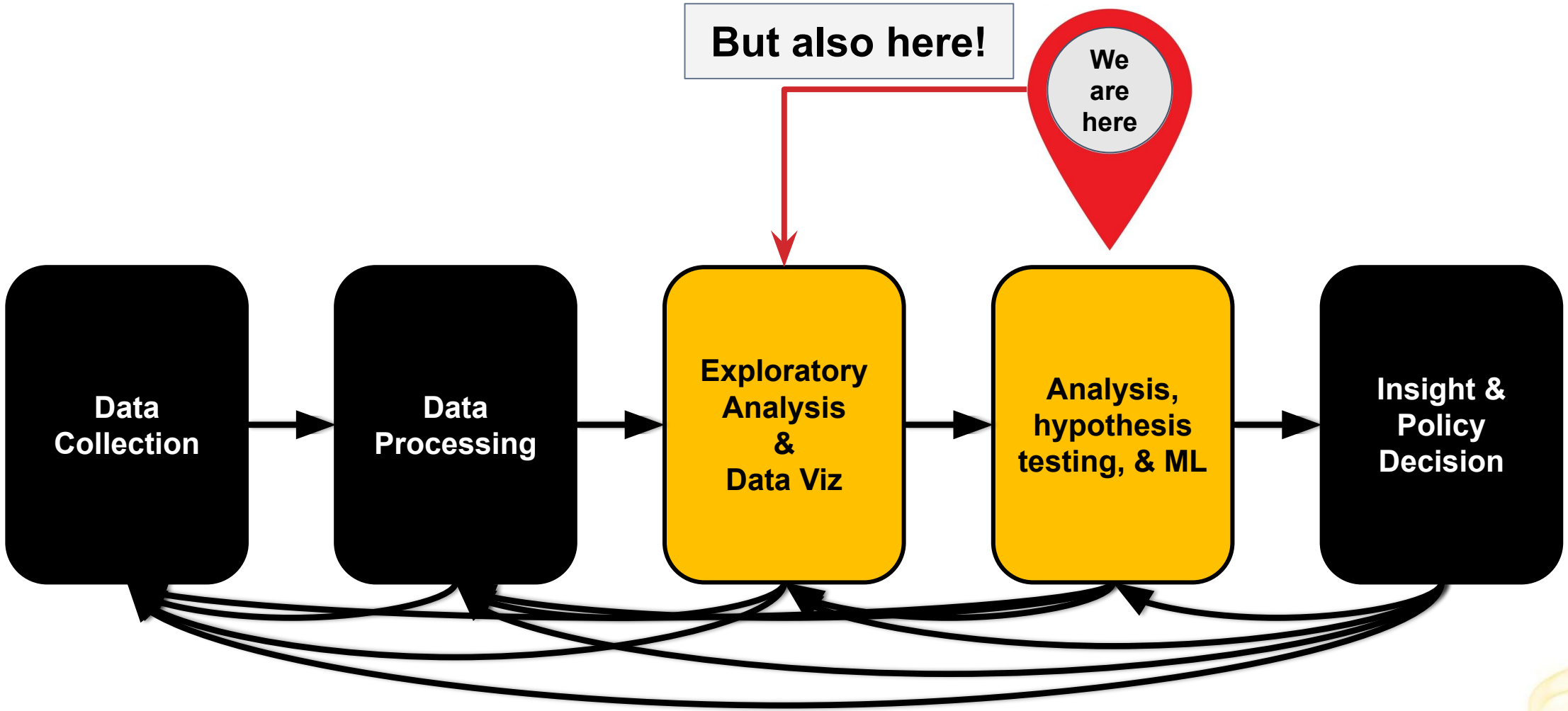
# Regularization Python Example

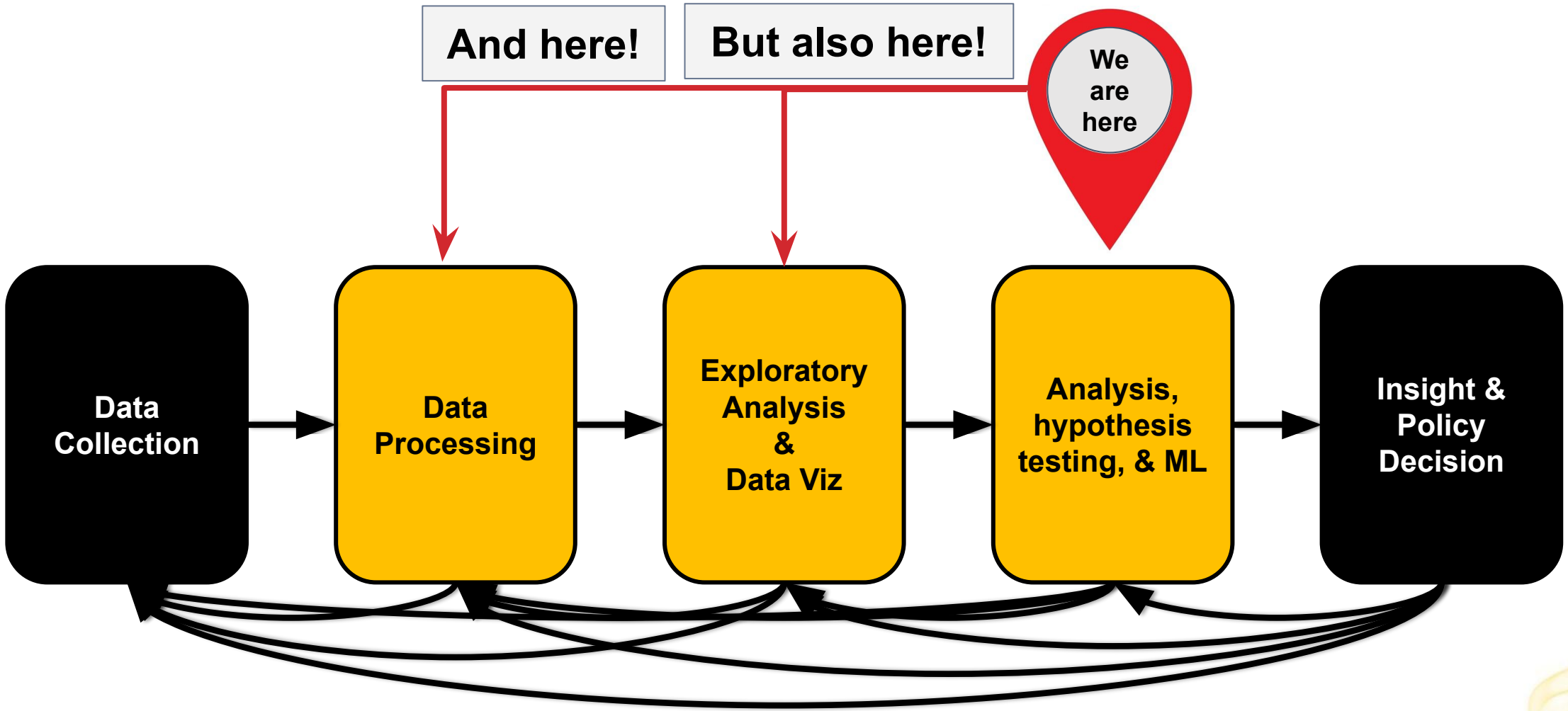
Note: In sklearn lambda is called alpha and is set to 1.0 by default.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# For all 3, create a model, train it, then predict on the test set

model_ridge = Ridge()
model_ridge.fit(X_train, y_train)
y_pred_ridge = model_ridge.predict(X_test)

model_lasso = Lasso()
model_lasso.fit(X_train, y_train)
y_pred_lasso = model_lasso.predict(X_test)

model_hybrid = ElasticNet()
model_hybrid.fit(X_train, y_train)
y_pred_hybrid = model_hybrid.predict(X_test)

# Print out the R2 score
print('Ridge:', model_ridge.score(X_test, y_test))
print('LASSO:', model_lasso.score(X_test, y_test))
print('Elastic-Net:', model_hybrid.score(X_test, y_test))
```

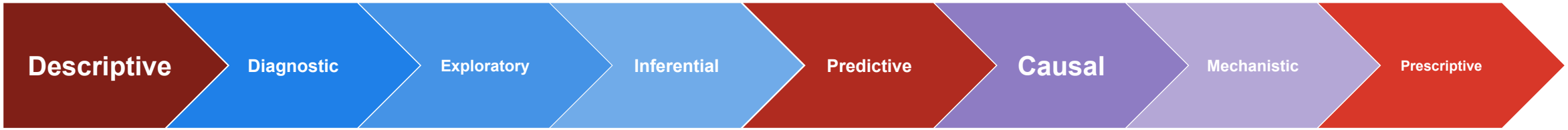
# Unsupervised Learning


We are here
Data Collection
Data Processing
Exploratory Analysis & Data Viz
Analysis, hypothesis testing, & ML
Insight & Policy Decision


But also here!
We are here
Data Collection
Data Processing
Exploratory Analysis & Data Viz
Analysis, hypothesis testing, & ML
Insight & Policy Decision


And here!
But also here!
We are here
Data Collection
Data Processing
Exploratory Analysis & Data Viz
Analysis, hypothesis testing, & ML
Insight & Policy Decision

# Realms of Data Science Analysis




What happened

Why did it happen

When did it happen

Discover patterns and relationships you didn't know about

What will happen

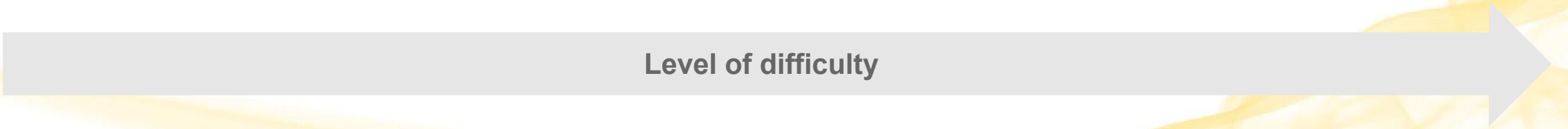
Why will it happen

When will it happen

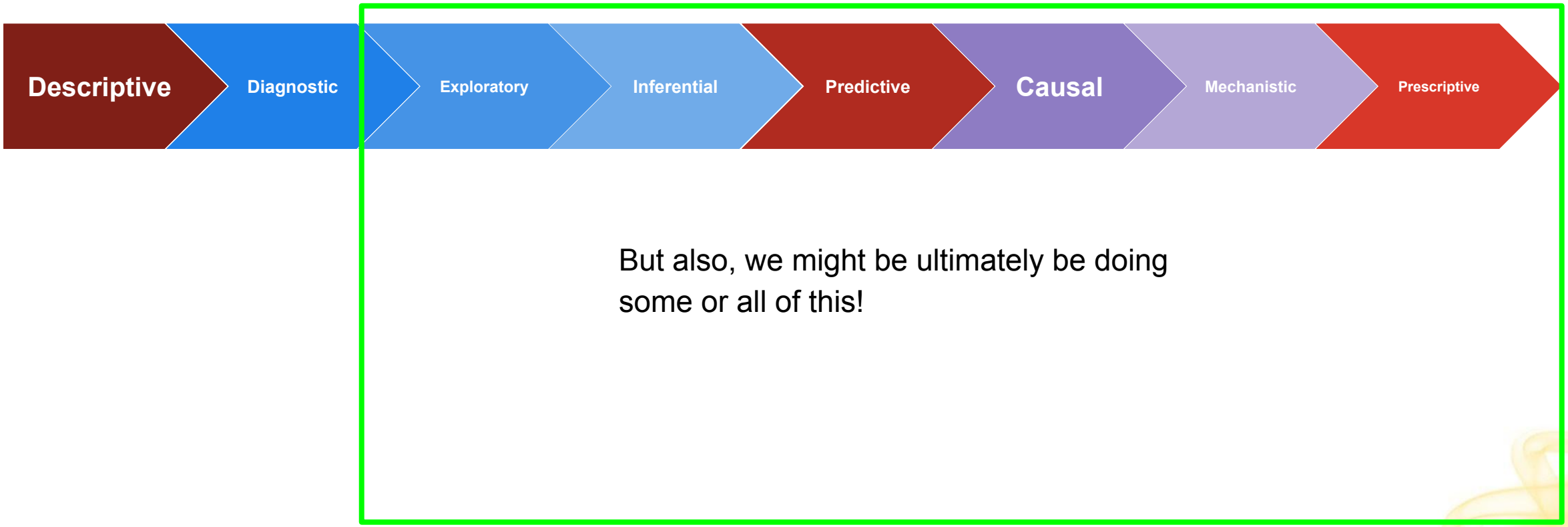
How can we make it happen

Why can we make it happen

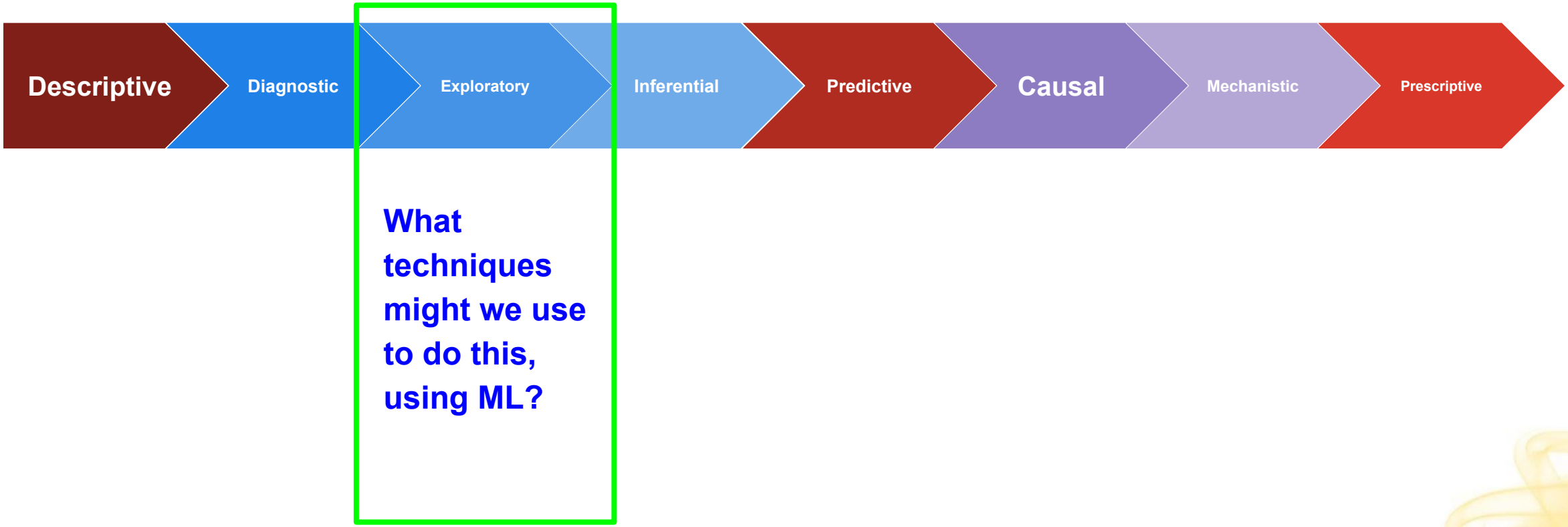
When can we make it happen



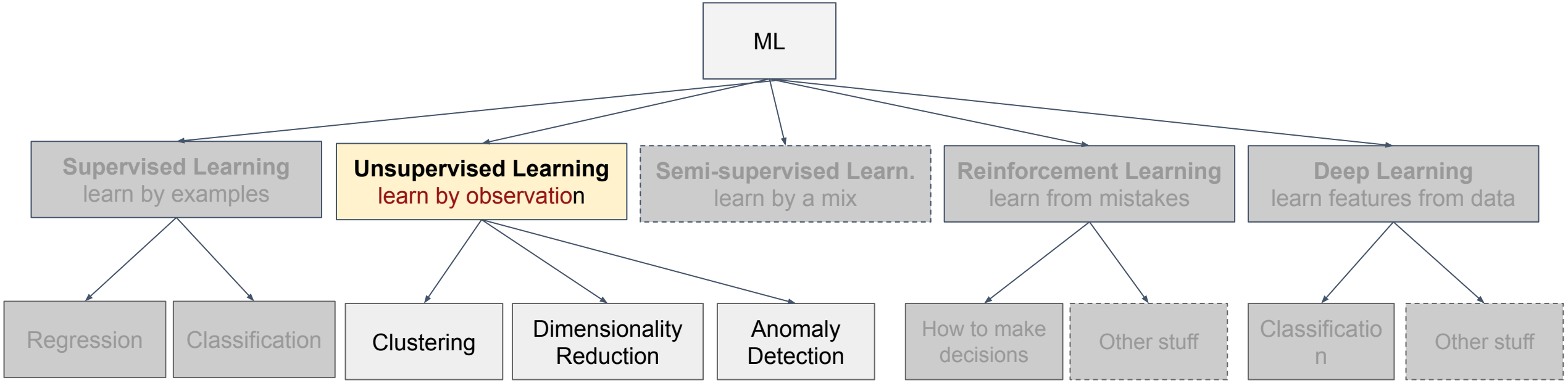
# Realms of Data Science Analysis



# Realms of Data Science Analysis



# Main categories of Machine Learning



# So far

What we've seen is that **Supervised** Machine Learning algorithms create models from data with **known** labels.

In other words, the data had target variable(s) with specific, known values that we could use to **train our models** such that we could **learn by example.**

# Real-World Problems

However, oftentimes data **does not come** with **predefined labels**.

**Problem**: We don't know the output, and we want to better understand patterns, structure, and relationships in our inputs.

**Solution**: Develop machine learning models that can autonomously **identify intrinsic patterns and structures** in unlabeled data.

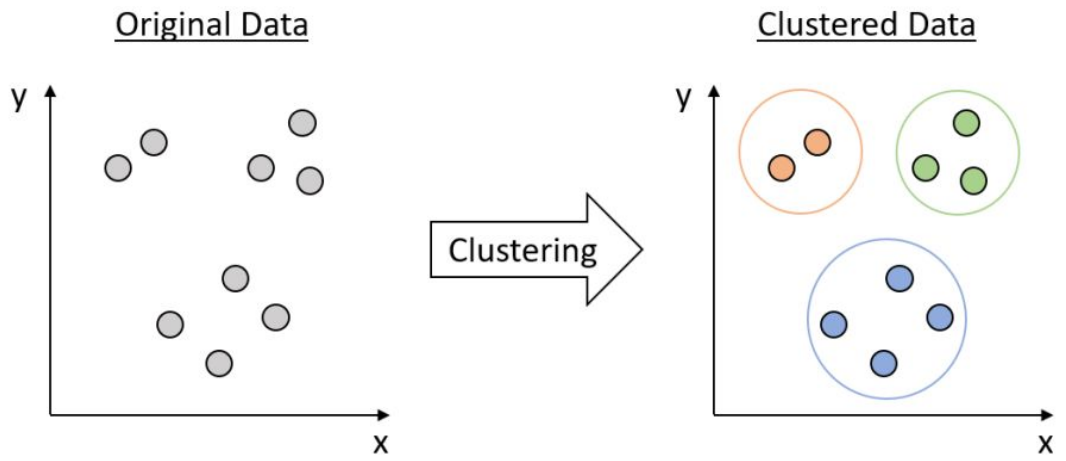
- In other words, **learn by observation**.

**This technique is called Unsupervised Learning.**

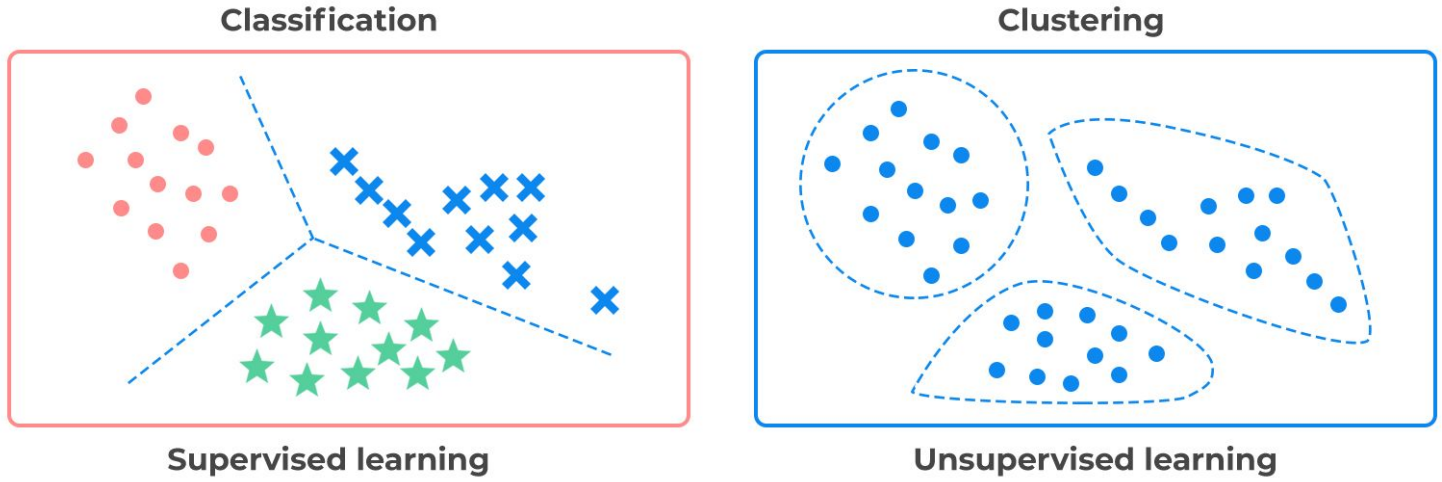
# Definition: Unsupervised Learning

Unsupervised Learning uses machine learning algorithms to analyze, explore, and group **unlabeled data**.

- These algorithms discover hidden patterns and structures without the need for human intervention.
- Ex. Clustering




The main goal of these types of algorithms is to study the **intrinsic structure of the data** in order to learn patterns, extract meaningful insights, and enable other work.




| | Supervised Learning | Unsupervised Learning |
|---|---|---|
| **Training Data** | Labeled (output/class is known) | Unlabeled (output/class is unknown) |
| **Objective** | Learn an unknown function that maps inputs to outputs | Learn hidden patterns or structures in or among the inputs |
| **Output** | Predicted output/class | Patterns or groupings |
| **Learning Method** | By example | By observation |

# Utility of Unsupervised Learning

Enables us to discover **similarities among patterns** and **departures from expected patterns/characteristics**

- Helpful for finding groupings or patterns when we don't know them
- Helpful for detecting anomalies and bad actors
- Helpful for figuring out when we need to modify our approach
  - ex. try a different classifier
- Helpful for figuring out which features might be useful/most useful

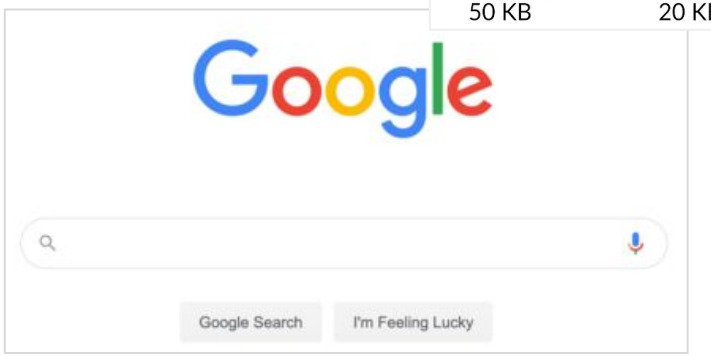
Can help us get **more value** for our spending ("more bang for our buck")

- Creating labeled datasets can be expensive, in terms of time and money 💰
- One way to manage this: build a supervised model using a small-ish set of labeled training data, then modify it to run unsupervised on a larger, less expensive, unlabeled dataset
- Another approach: use an unsupervised algorithm to find groupings and label them, then use that labeled data to train a supervised model.

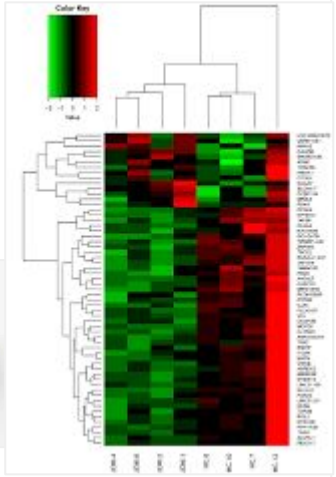
# Real-World Applications of Unsupervised Learning

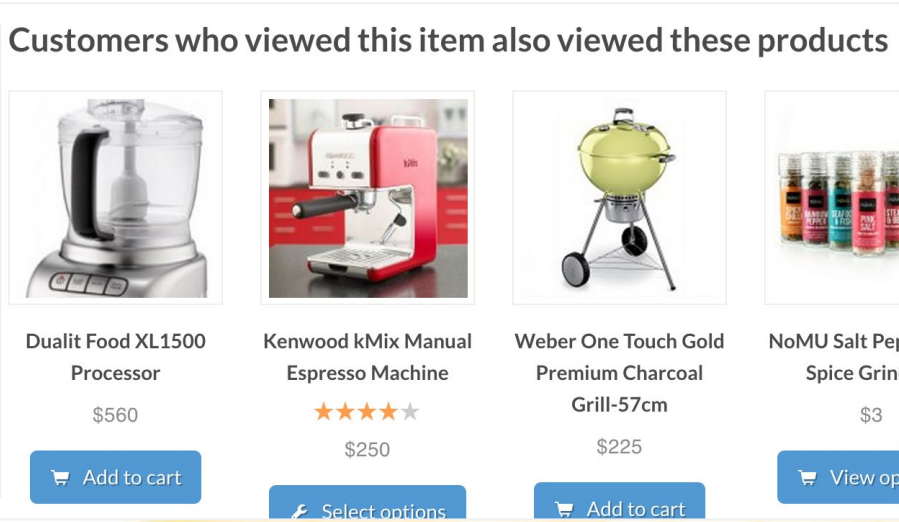
Some Examples:

- Customer segmentation
- Fraud detection
- Recommendation engines
- Compression
- Bioinformatics applications like
  - Protein structure prediction
  - Gene clustering
- NLP applications like search indexing
  - Semantic understanding
  - Document clustering
  - Topic modeling
  - …many more







# Core Areas of Unsupervised Learning

## Clustering

- Segment datasets by shared attributes to identify **natural groupings** (clusters) within the data.

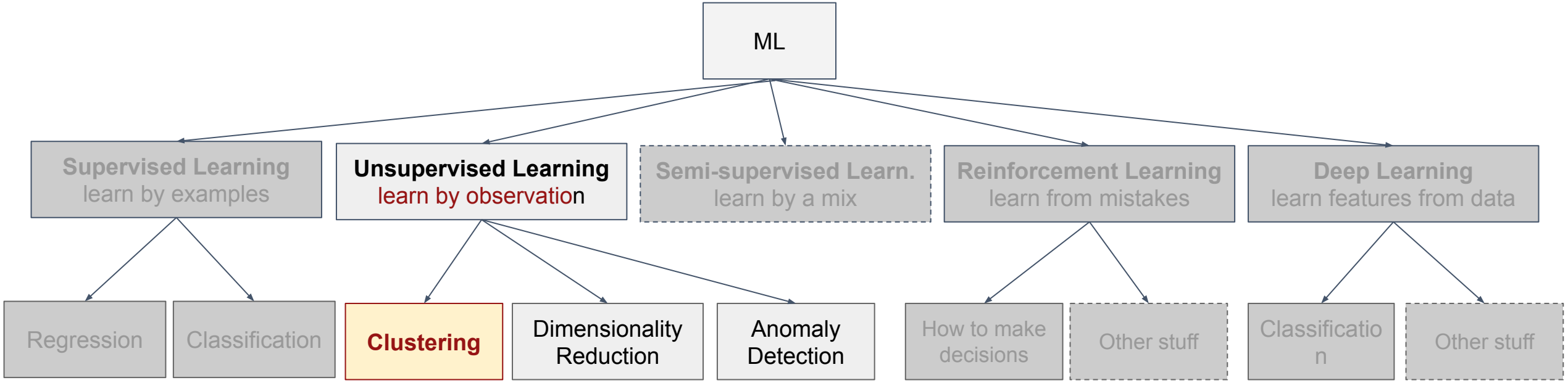
## Dimensionality Reduction

- Simplify datasets by aggregating variables (features) with similar attributes, in order to **reduce the number of features** (dimensions) while preserving the essential structure of the data.

## Anomaly Detection

- Identifying **anomalies** in the data that do not conform to the patterns exhibited by the majority of the dataset.

# Main categories of Machine Learning



# Clustering

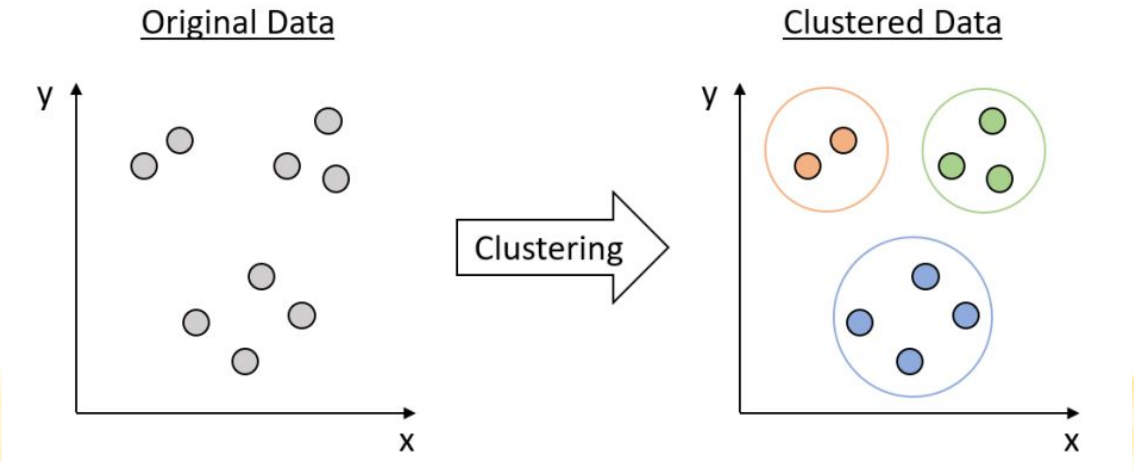
# What can we cluster in practice?

- News articles or web pages by topic
- Protein sequences by function, or genes according to expression profile
- Users of social networks by interest
- T-shirts by small, medium, and large sizes
- Customers by occasional buyer and frequent buyers
- Network traffic by expected vs unexpected behavior
- Patients by seriousness of illness
- Physical regions by high incidence of a particular disease (ex. covid outbreak)
- …many, many more things.

# Clustering

Also called cluster analysis, clustering is a powerful unsupervised learning technique used to identify natural groupings (cluster) within datasets.

- **Group similar data points** or objects into clusters or categories.
- By grouping data points with similar characteristics, cluster analysis helps **reveal hidden patterns, trends, and relationships**.



# What do we mean by similar?

Similarity can be hard to define, but we know it when we see it.

# Similarity vs Dissimilarity

- **Similarity**
  - Numerical measure of how “alike” two data points are.
  - Higher when they are more alike.
  - Usually in the range [0,1] (0=no similarity, 1=total similarity)
- **Dissimilarity**
  - Numerical measure of how “different” two data points are
  - Lower when they are more alike
  - Minimum is often 0 (no dissimilarity/total similarity)
  - Maximum varies by measure
- **Proximity** refers to a similarity or dissimilarity
  - Usually measured by a **distance** function

# Common Proximity Measures

- Euclidean Distance (L2 norm)
- Manhattan Distance (L1 norm)
- Hamming Distance
- Cosine Similarity
- Jaccard Similarity
- Pearson Correlation
- Chi-Square

# Similarity in cluster analysis

In clustering, the definition of similarity **depends on the technique you're using**.

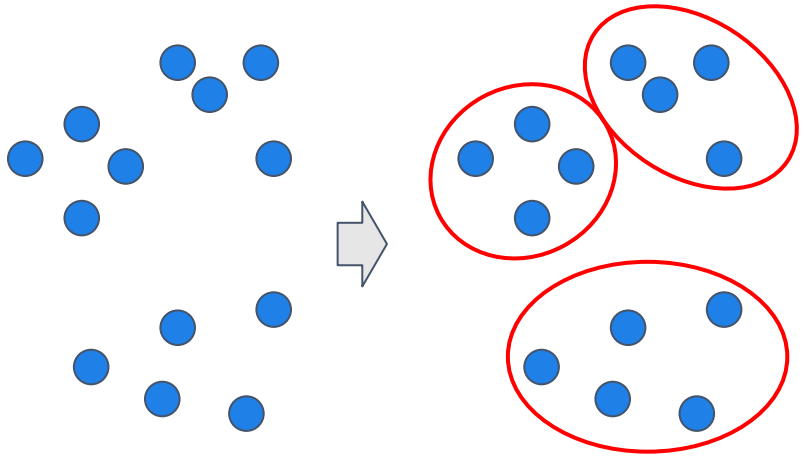
In general, we use measures of **proximity** (distance) specific to the problem at hand. Some examples:

- **Centroid-based clustering**
  - Proximity is defined by **distance from a centroid**
  - The centroid is the cluster **center**, which is the **mean** of all data points in the cluster.
- **Hierarchical clustering** (HCA)
  - Proximity is **distance between data points**
  - Also considers **distances between clusters**
- **Density-based clustering**
  - Proximity is **distance between data points in data space** (a contiguous region of high data point density)
    - Basically how densely packed together data points are

# Types of Clustering

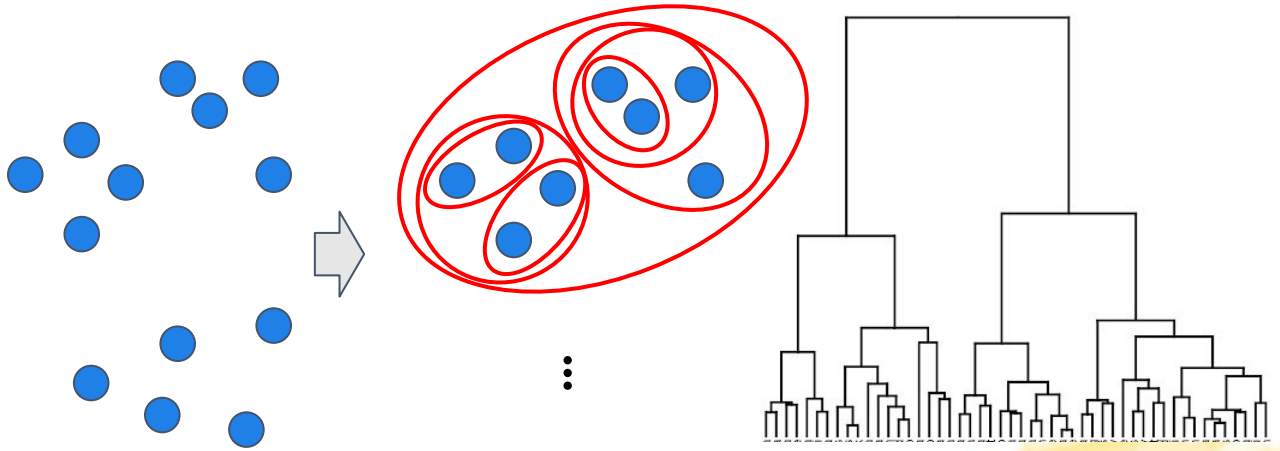
## 1. Partitional Clustering

Data is partitioned into non-overlapping subsets (clusters) such that each data point is in exactly one subset



## 2. Hierarchical clustering

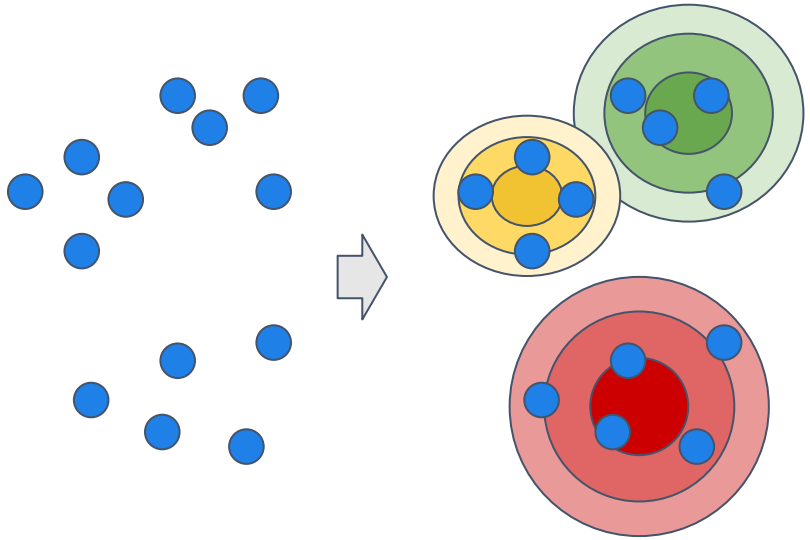
A set of nested clusters is organized hierarchically as a tree



# Types of Clustering

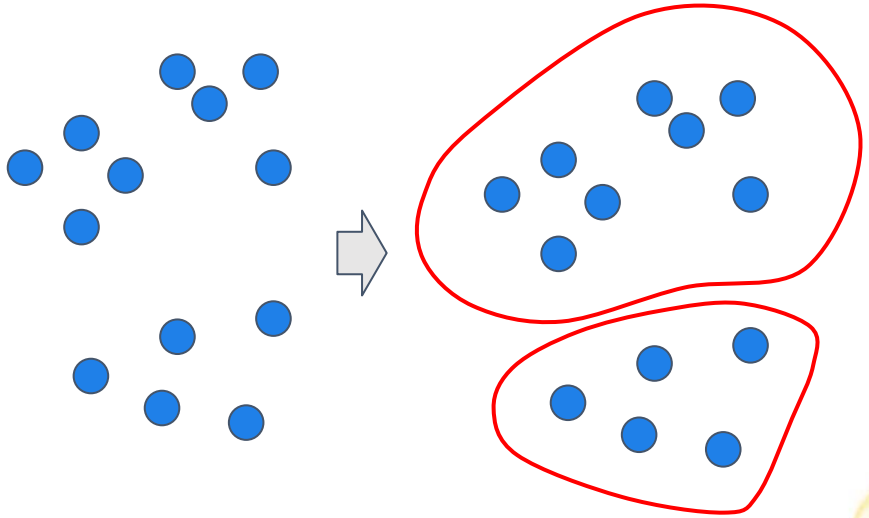
### 3. Model-Based Clustering

Data is clustered into homogeneous groups using probability distributions.



### 4. Density clustering

Data is grouped by areas of high density separated by areas of low density.



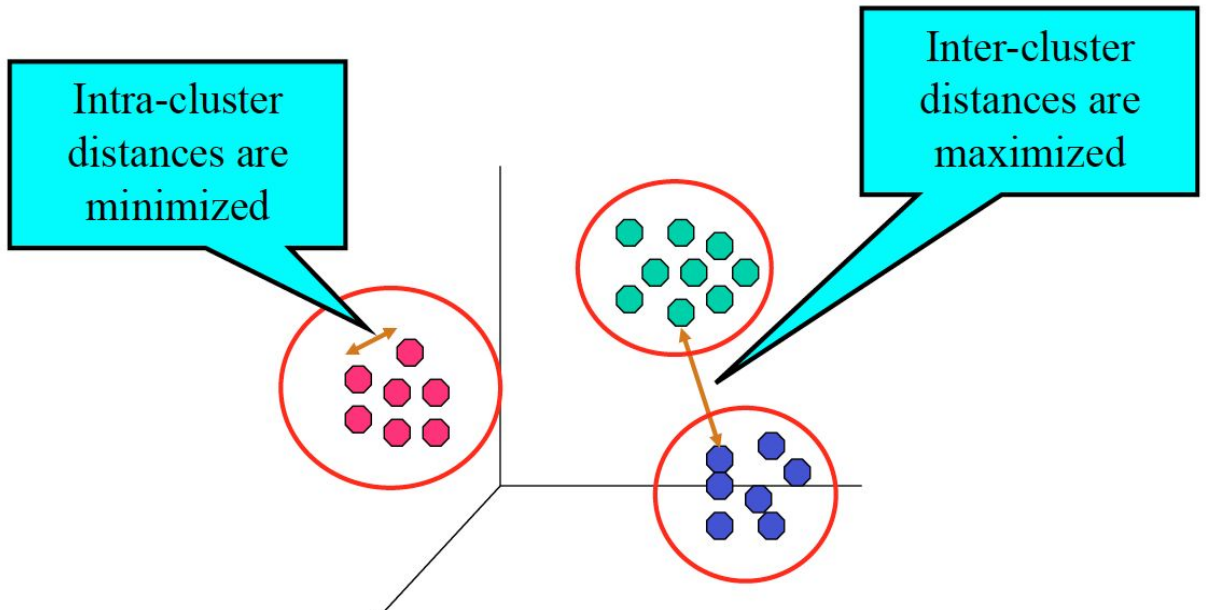
# Clustering Objectives

**Find groups such that objects within the same group are similar, while being different from objects in other groups.**

- **Intra-Cluster Distances are Minimized:** Group observations that are very similar or homogeneous together in same cluster.
- **Inter-Cluster Distances are Maximized:** Observations belonging to different clusters are different or heterogeneous.

Want high intra-cluster similarity

"Cohesion"




Want low inter-cluster similarity

"Separation"

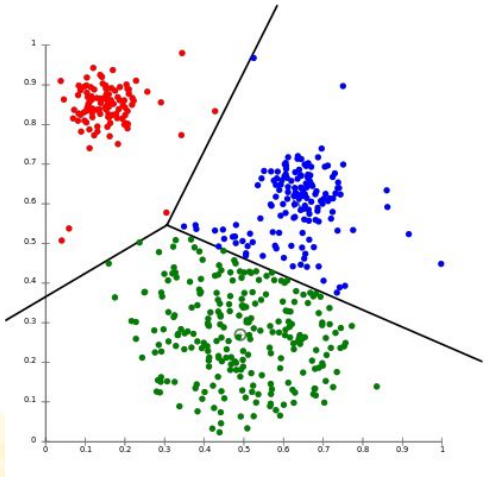
# Clustering Inputs and Outputs

**Input**:

- $n$ data points in feature space
- a distance measure $d$

**Output**:

- A set of clusters (groups, or partitions) where data points are grouped based on their similarities



# Most Common Clustering Algorithms

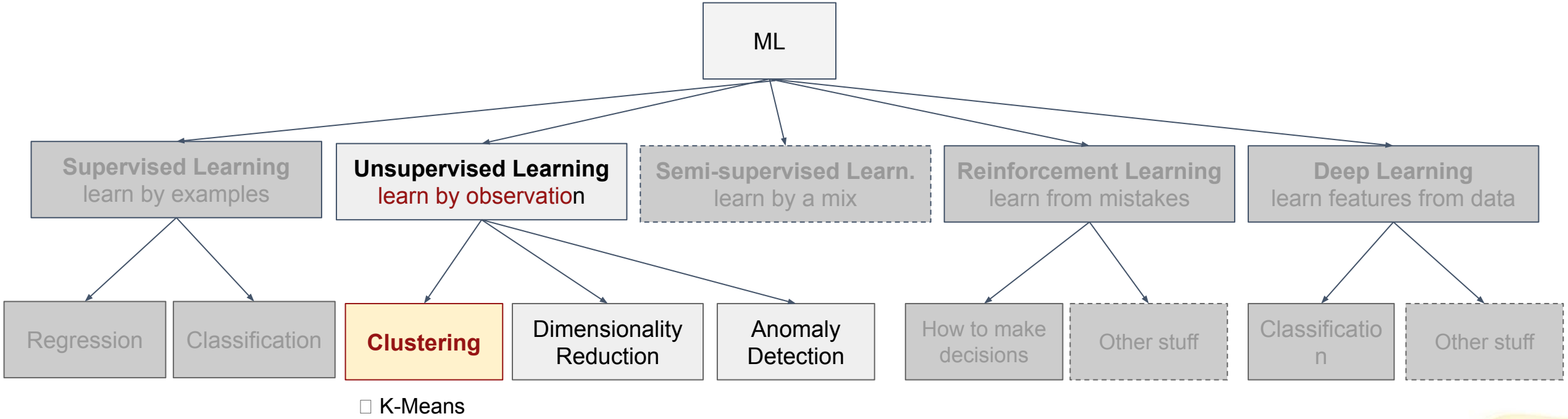
- **K-Means** - centroid-based clustering
- **Agglomerative** (AGNES) - hierarchical clustering
- **Density Based Scan Clustering** (DBSCAN) - density clustering
- **Gaussian Mixture Model** - probabilistic model-based clustering

# Most Common Clustering Algorithms

We'll just look at this one

- **K-Means** - centroid-based clustering
- **Agglomerative** (AGNES) - hierarchical clustering
- **Density Based Scan Clustering** (DBSCAN) - density clustering
- **Gaussian Mixture Model** - probabilistic model-based clustering

# Main categories of Machine Learning



# K-Means Clustering

# K-Means

- An **unsupervised learning** algorithm
- Most popular clustering algorithm
- Centroid-based, partitional clustering
- We'll review in 2 dimensions for illustration, but K-Means applies to higher dimensions as well

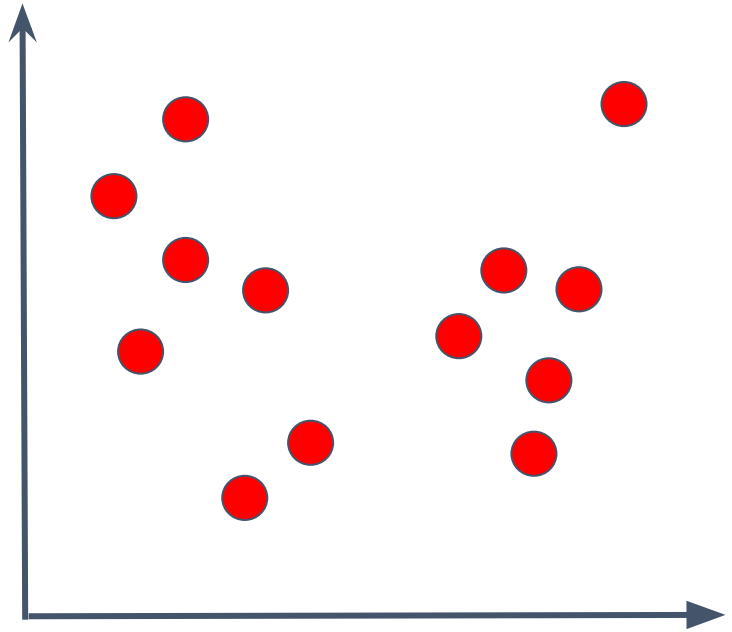
# K-Means Clustering Algorithm steps

**Algorithm 1** $k$-means algorithm

1: Specify the number $k$ of clusters to assign.
2: Randomly initialize $k$ centroids.
3: **repeat**
4: **expectation:** Assign each point to its closest centroid.
5: **maximization:** Compute the new centroid (mean) of each cluster.
6: **until** The centroid positions do not change.

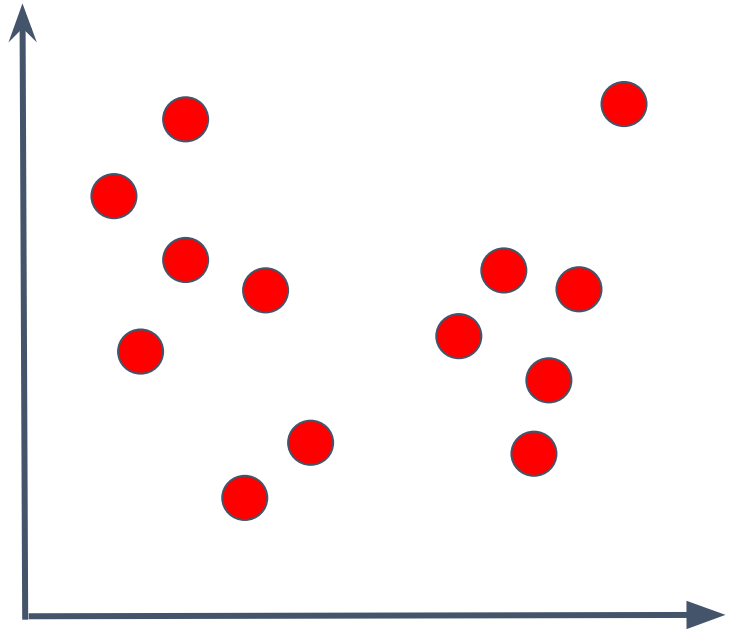
# K-Means Clustering Visualized

Suppose we have 2 features and the following observations:



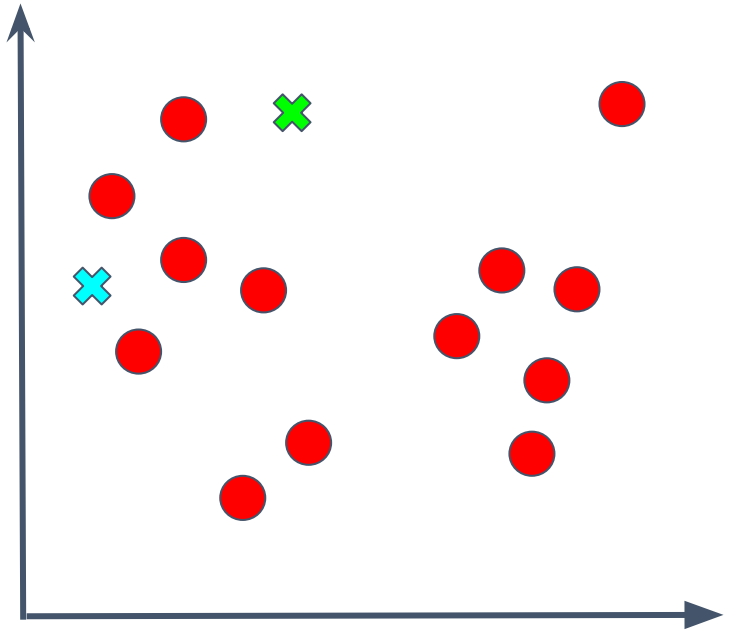
# K-Means Clustering Visualized

**Step 1**: Choose k (*more on this later*). We'll pick k=2 for this example.



# K-Means Clustering Visualized

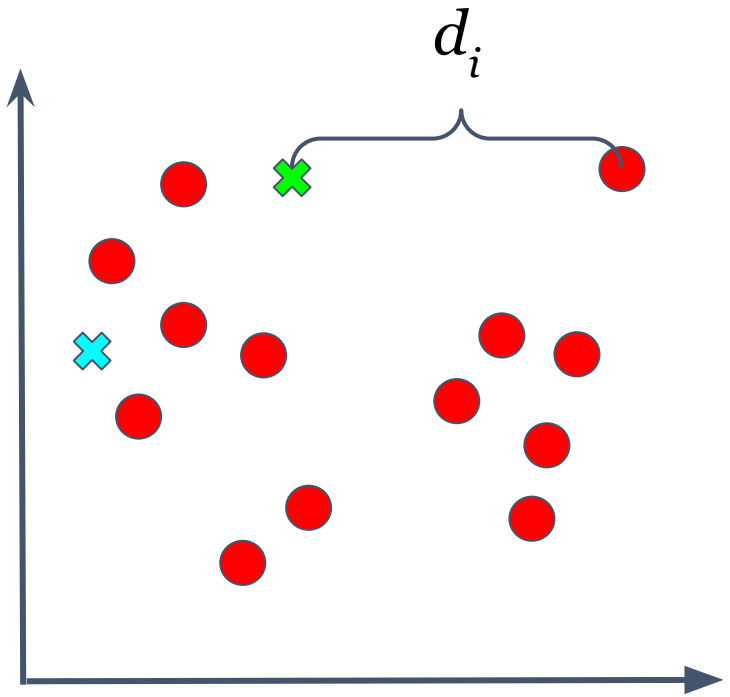
**Step 2**: Randomly create k=2 centroids:



# K-Means Clustering Visualized

**Step 3**: Assign each data point its “closest” centroid.

We can use **Euclidean Distance** as the distance function for our proximity metric.



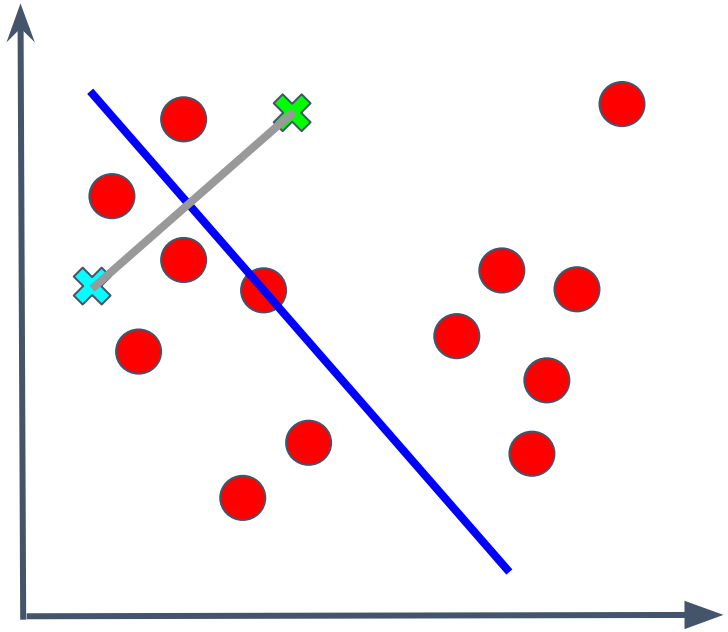

Simply calculate the distance of each datapoint to each centroid, and assign accordingly.

$$d(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

# K-Means Clustering Visualized

**Step 3**: Assign each data point its closest centroid.

We can use **Euclidean Distance** as the distance function for our proximity metric.



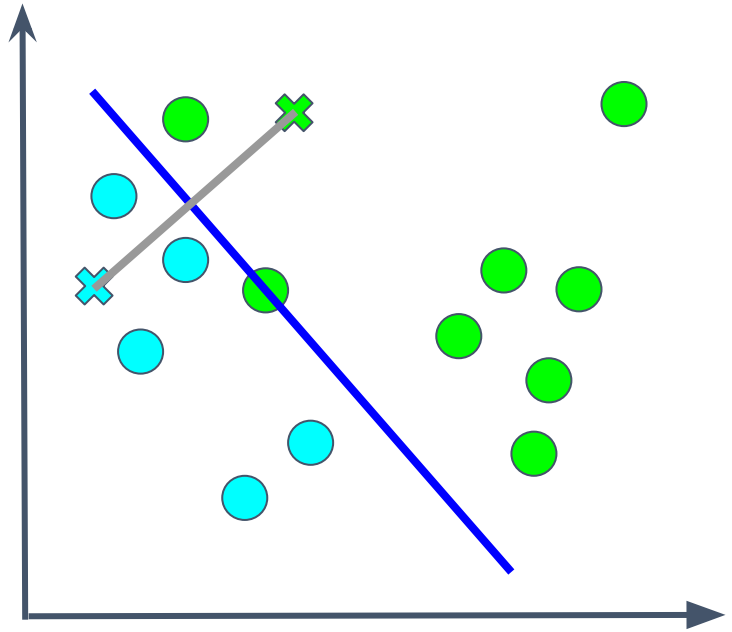
<u>To visualize</u>:

1. Draw a line bisecting the centroids
2. Any point on the line is equal distance between the 2 centroids (pick one randomly)
3. Points on either side of the line are closer to the corresponding centroid

# K-Means Clustering Visualized

**Step 3**: Assign each data point its closest centroid.

We can use **Euclidean Distance** as the distance function for our proximity metric.



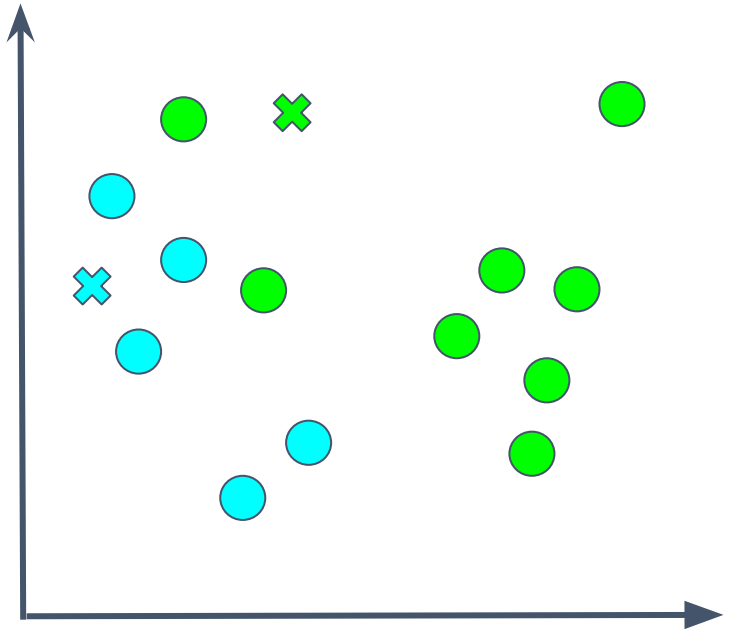
<u>To visualize</u>:

4. Assign data points to centroids accordingly.

# K-Means Clustering Visualized

**Step 3**: Assign each data point its closest centroid.

We can use **Euclidean Distance** as the distance function for our proximity metric.

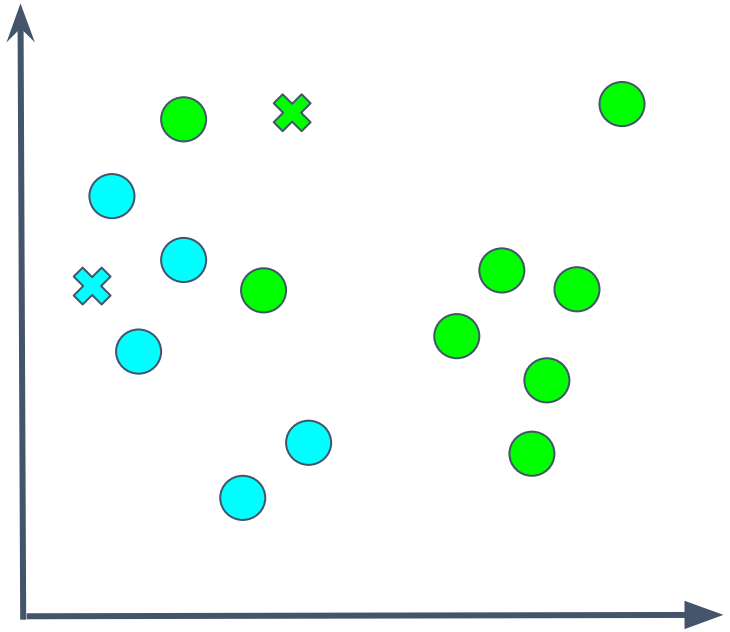


<u>To visualize</u>:

4. Assign data points to centroids accordingly.
5. These are our “clusters” for the first iteration.

# K-Means Clustering Visualized

**Step 4**: Compute a new centroid (mean) for each cluster.

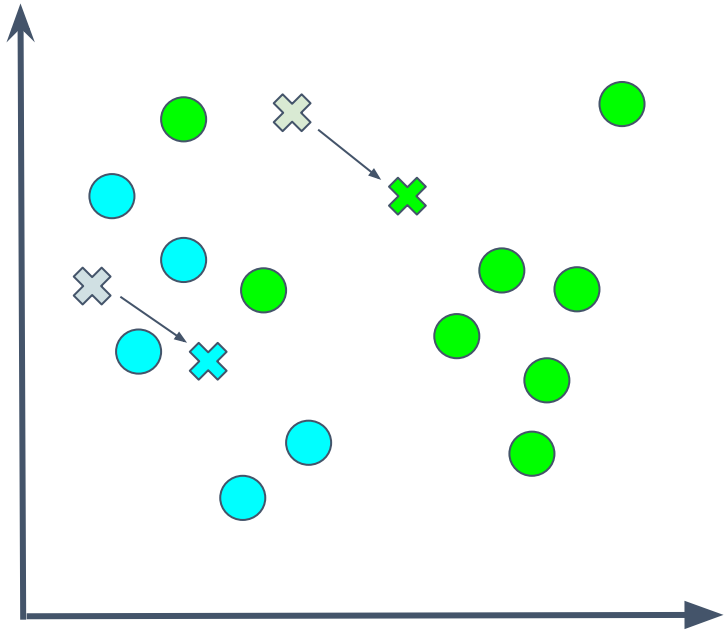


Simply calculate the mean of each cluster, and make that the new centroid.

$$C_i = \mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

# K-Means Clustering Visualized

**Step 4**: Compute a new centroid (mean) for each cluster.

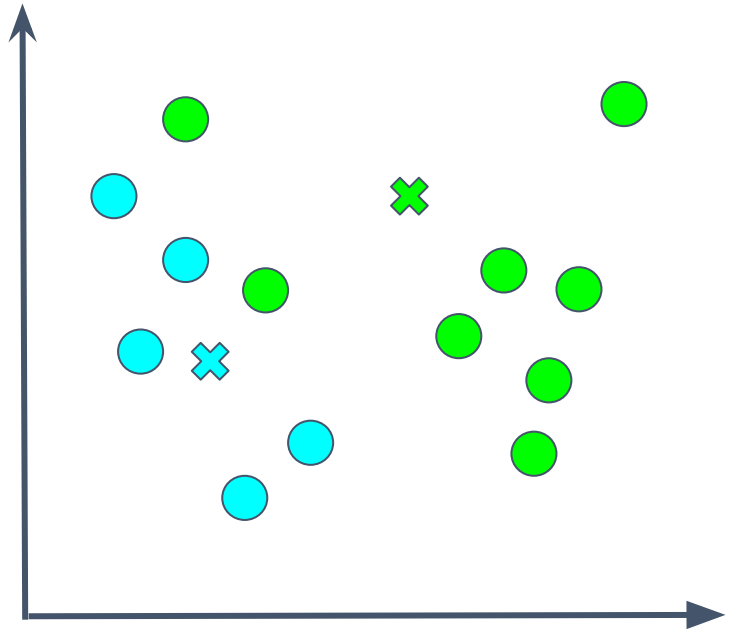


Simply calculate the mean of each cluster, and make that the new centroid.

$$C_i = \mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

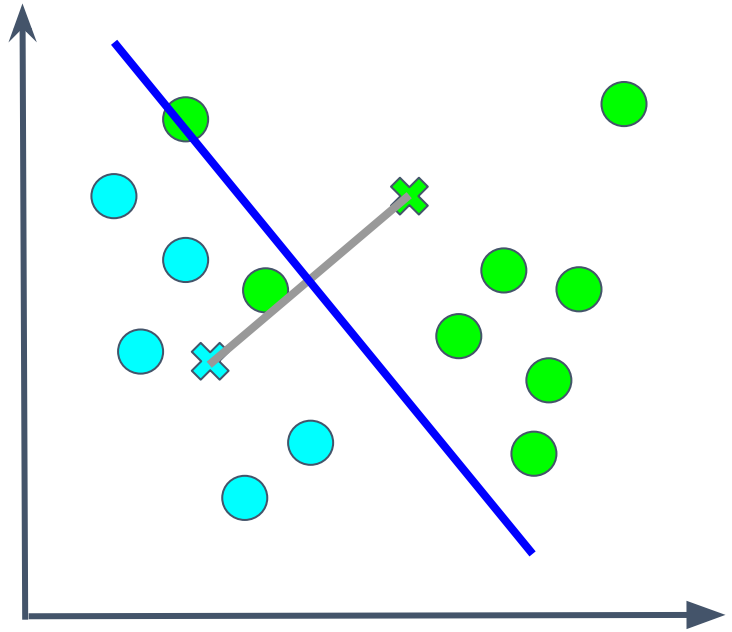
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



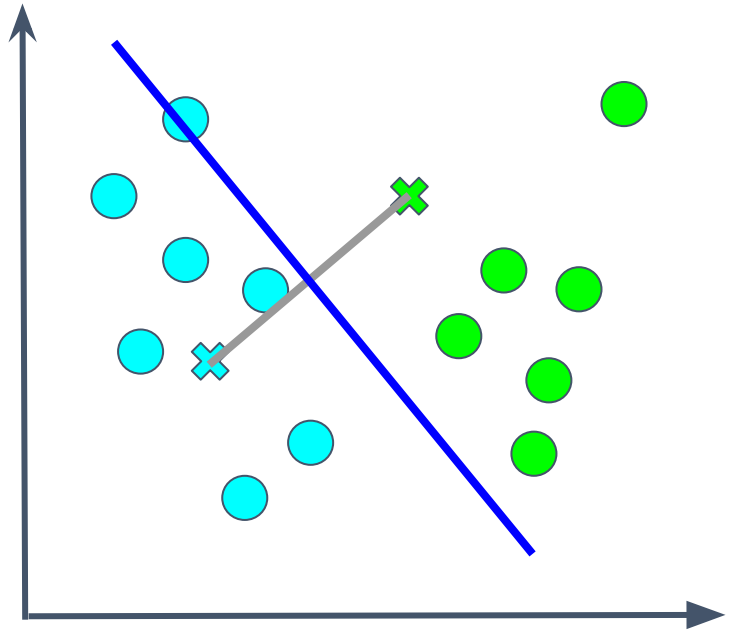
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



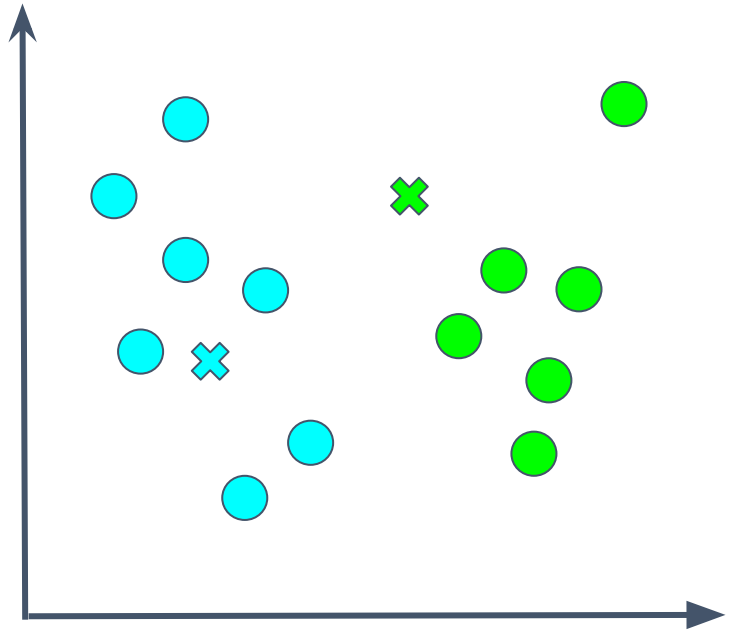
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



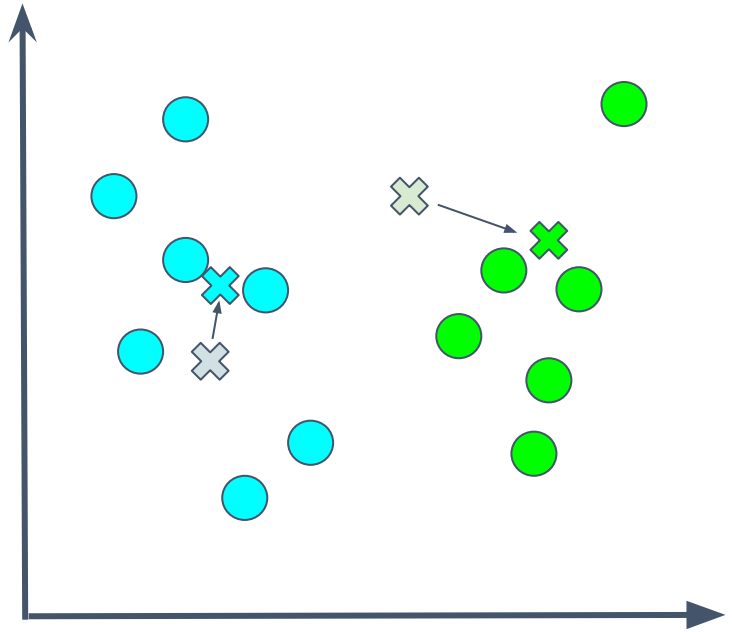
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



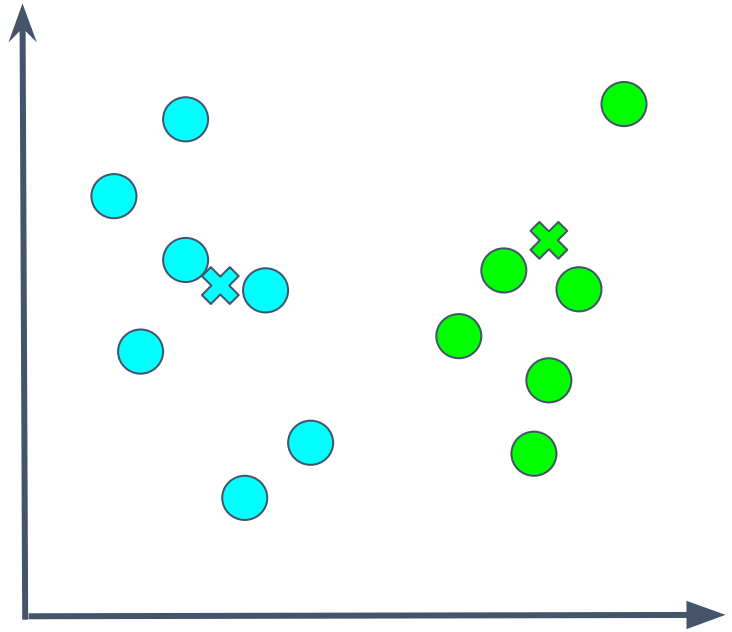
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



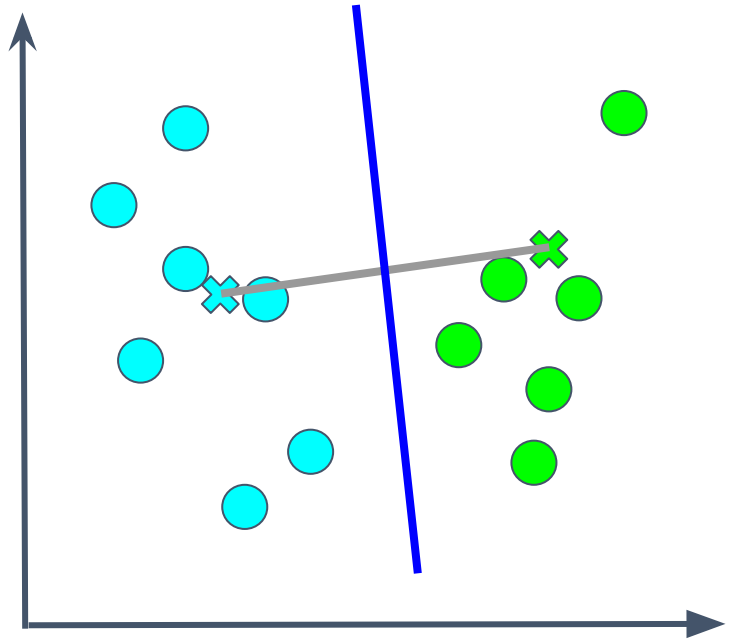
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



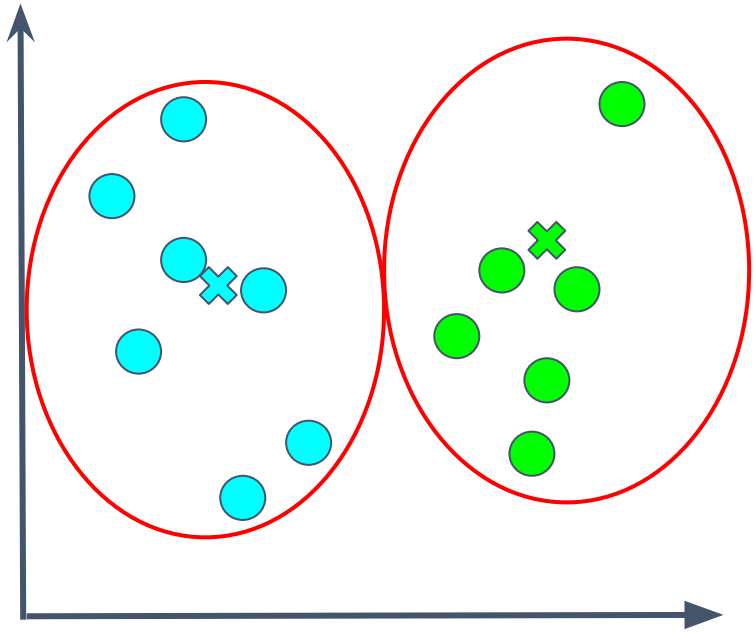
# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.



# K-Means Clustering Visualized

**Step 5**: Repeat until centroid positions stop changing.

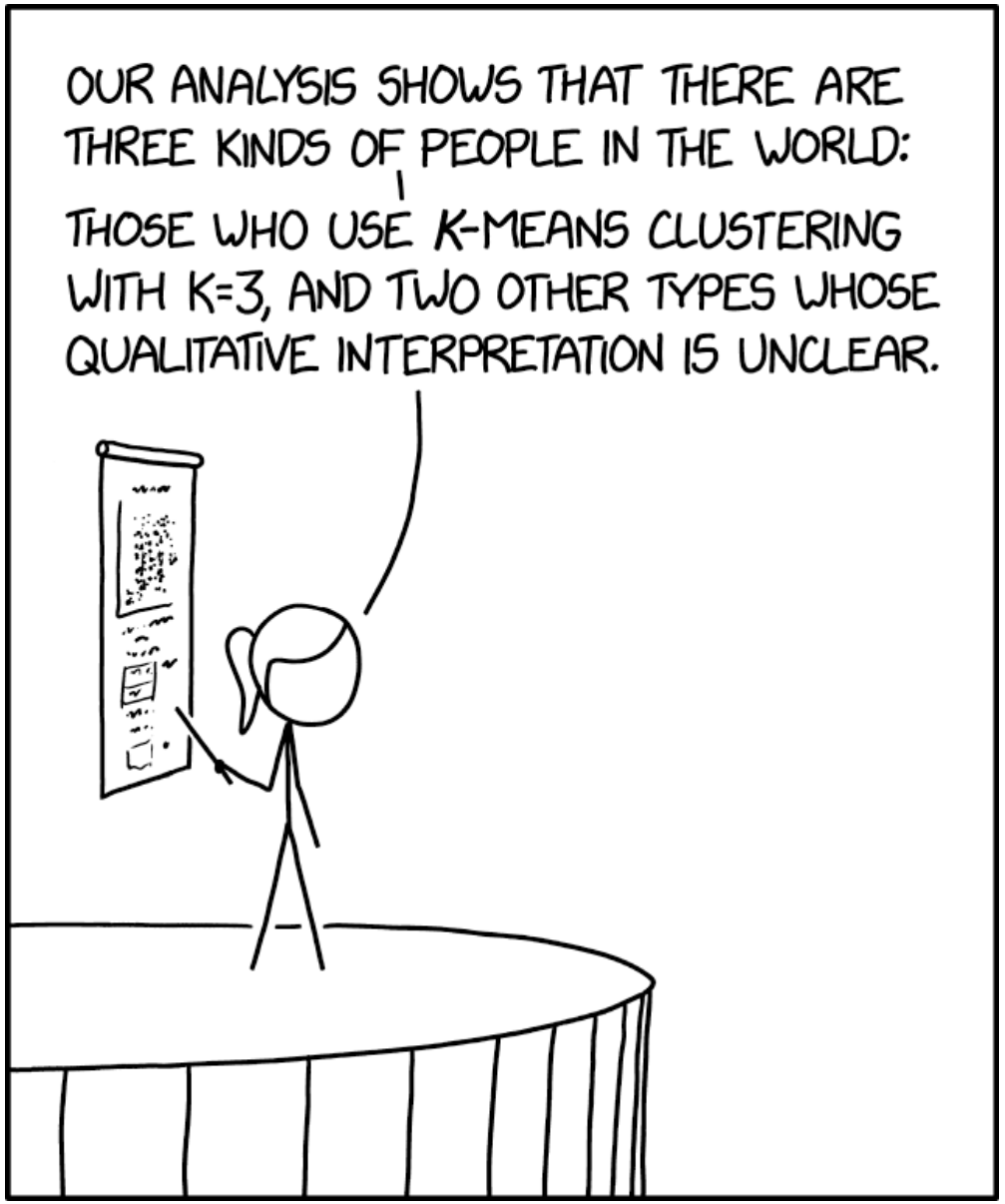


# Stopping Criteria


If any of these are true, stop:

1. **Centroids of the newly formed clusters do not change**
   - Not learning any new pattern
2. **All data points stay in their same cluster**
   - No change to clusters
3. **Minimum amount of error reached** *(more on this in a min)*
   - Calculate Residual Sum of Squares (RSS, aka SSE) and check if small
4. **Maximum number of iterations are reached**
   - You hit a maximum number of iterations you want to run (usually < 50)

How do we **choose k**?


OUR ANALYSIS SHOWS THAT THERE ARE THREE KINDS OF PEOPLE IN THE WORLD:
THOSE WHO USE K-MEANS CLUSTERING WITH K=3, AND TWO OTHER TYPES WHOSE QUALITATIVE INTERPRETATION IS UNCLEAR.

# How to choose hyperparameter k?

**If k=n**

- One cluster is created for *every* data point.
  - Extreme overfitting

**If k=1**

- One giant cluster is created for the *entire* data set.
  - Extreme underfitting

A good **k** will ensure clusters are distinct from each other, meaning:

- The distance from **each point to its centroid** is **much smaller** than the distance **between centroids** of different clusters.

# K-Means Clustering as an Optimization Problem

*Minimize total **intra-cluster** variance, or **Within-Cluster Sum of Squares (WCSS)***

*Where WCSS = sum of squares of the distances of each data point to its respective centroid for all clusters.*

**Optimization function for k-means clustering**

$$J = WCSS = \sum_{j=1}^{k} \sum_{i=1}^{n} || x_i^{(j)} - c_j ||^2$$

Where

$k$ = number of clusters
$n$ = number of data points in cluster $c_j$
$c_j$ = the centroid for cluster $j$

**Assumptions:**

- Each observation belongs to at least one of the k clusters
- Clusters do not overlap
- Variation within each cluster is minimized.

But, **finding the global minimum of $J$ is NP-hard**, so we need another way.

# Ways to find a good k

- **Domain Knowledge**
  - Sometimes we know from experience what a good k should be.
- **Cross-Validation**
  - Run K-Means with a variety of values for k and compare.
- **Elbow Method**
  - Just like we did with k-Nearest Neighbors, calculate the WCSS (Within-Cluster Sum of Squares) for every value of k and pick the elbow as your k value
  - Not very quantifiable/precise, but fast and sometimes good enough
- **Silhouette Method**
  - Gives us a quantifiable method for determining how well the data is clustered
  - But, be careful: Silhouette scores will penalize irregular or nested clusters, even when they make sense
  - We haven't looked at this one yet - let's take a peek! 👀

# Silhouette Method

- For all clusters:
  - For all data points in the cluster:
    - Calculate a score for the data point, indicating whether it is likely to be clustered well
    - Score Range: -1 to +1
      - +1 = Data point is clustered very well
      - - 1 = Data point is clustered poorly
      - 0 = Coin toss
  - Take the average across all data points in the cluster for the $j^{th}$ cluster's quality
- Take the average cluster quality across all clusters for the overall Silhouette Score
- Choose k that produces the **highest overall Silhouette Score**

# Silhouette Score for a single data point

To calculate the Silhouette Score for a **single data point**, remember we care about 2 things:

1. **Cohesion**: small distances between points in the same cluster
   - Take the average Euclidean distance between this $i^{th}$ data point and all other data points in the **same cluster**
   - Call this value $a_i$
2. **Separation**: large distances between points in different clusters
   - For all clusters this data point is not in:
     - Take the average Euclidean distance between the $i^{th}$ data point and all data points in this **different cluster**
   - Take the minimum of all these cluster averages (represents the worst case - the closest cluster)
   - Call this value $b_i$

Silhouette Score = $b_i$ - $a_i$ normalized onto [-1, 1] :

$$\frac{b_i - a_i}{max(b_i, a_i)}$$

# Silhouette Score Python Example

```python
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv('customers.csv')
X = df.drop(['ID'],axis=1)

# Calculate silhouette score for various k
scores = []
possible_k = range(2, 15)
for k in possible_k:
  model = KMeans(n_clusters = k)
  model.fit(X)
  scores.append(silhouette_score(X, model.labels_, metric = 'euclidean'))

# Pick the best score
best_k = scores.index(max(scores)) + min(possible_k)
print('Best k value to use is', best_k)
```

```
Best k value to use is 2
```

```python
# Plot it so we can see visually
plt.plot(possible_k, scores, 'bx-')
plt.xlabel('k')
plt.ylabel('Silhouette Score')
plt.show()
```